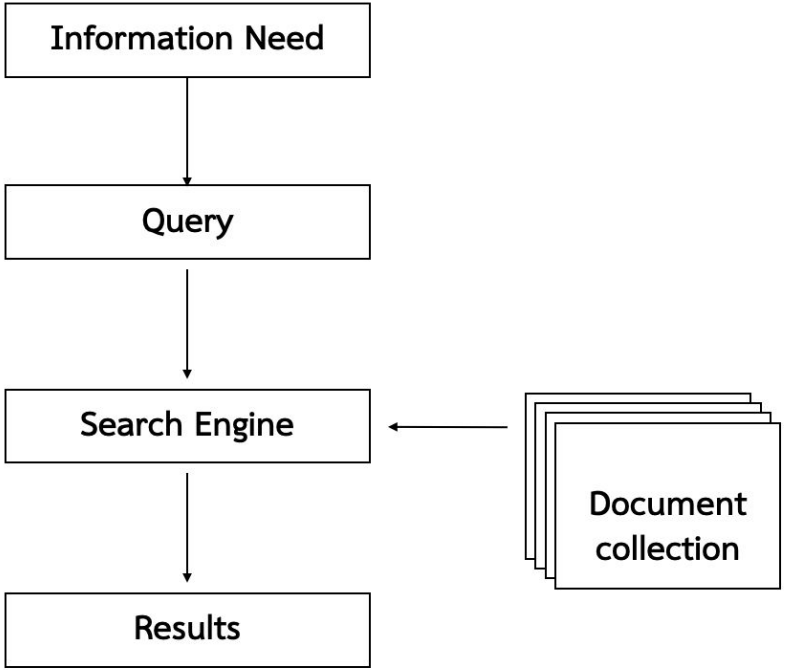
Advanced IR and RAG

NLP II 2026

Assoc. Prof. Attapol Thamrongrattanarit

Classic search model

What does intrinsic evaluation actually evaluate?



Which is the better metric?

Clickthrough Rate
(CTR)

อัตราการสั่งอาหาร
อัตราการจองโรงแรม
อัตราการสั่งซื้อสินค้า

A/B testing

- A/B testing is a method of comparing the performance of two or more versions of a search engine, typically with the goal of improving the user's experience.
- In an A/B test, users are randomly assigned to one of two or more groups, each of which is shown a different version of the search engine. The performance of each version is then compared based on metrics such as click-through rate, conversion rate, or time spent on the site.
- A/B testing can be used to evaluate all aspects of the search engine, such as the ranking algorithm, the search results page layout, word segmentation, or the wording of the search query box.

Steps to A/B Testing

- Evaluate the new system(s) on click data
- Divert small amount of traffic to the new system e.g.
  - 1% new system
  - 99% status quo
- Wait and monitor
- Roll out to more

Wait and monitor

- Compute the metrics that we want
- We need to run the experiment for a long time
  - The traffic is small
  - The difference is usually small e.g. order rate from 0.50% to 0.51% might translate to thousands per year.
- How do we know the systems are significantly different?

A/B Testing

Pros

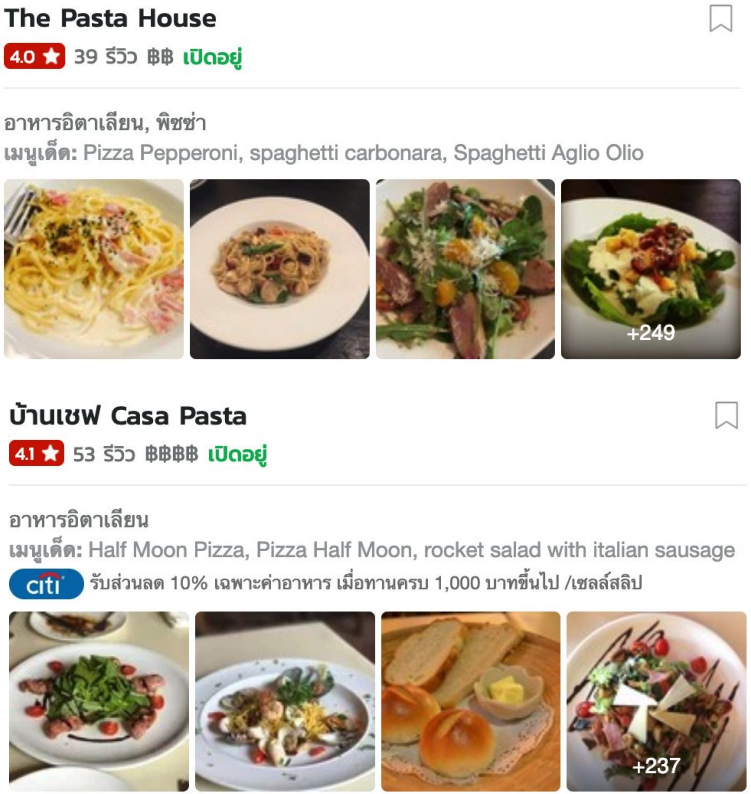
- It measures if we address the information need directly.
- We can use just any metrics that matter to us.

Cons

- It takes a lot of time.
- It requires a good infrastructure to redirect users.

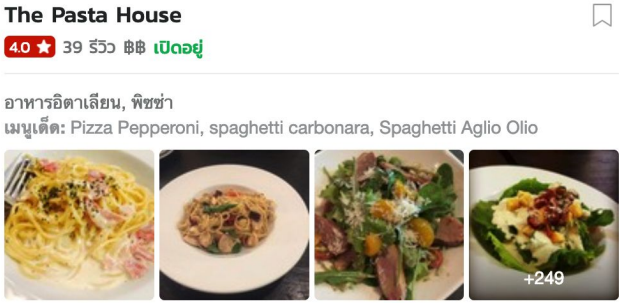
Query understanding

พาสต้า โรแมนติก สีลม ไม่แพง

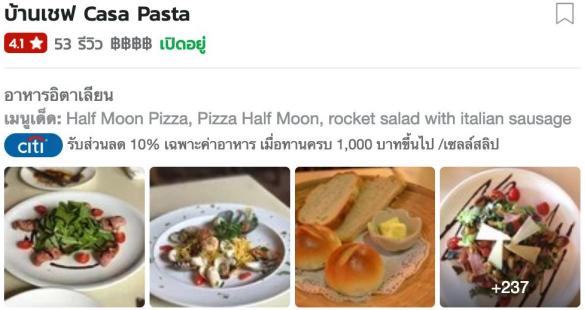


พาสต้า โรแมนติก สีลม ไม่ แพง

พาสต้า
Category:Italian
โรแมนติก
Location: สีลม
Attribute: ฿฿



Category:Italian
Location: สีลม
Attribute: ฿฿



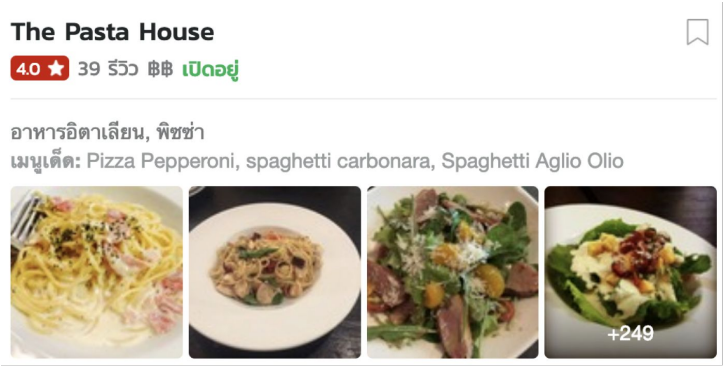
Category:Italian
Location: อุดมสุข
Attribute: ฿฿฿฿

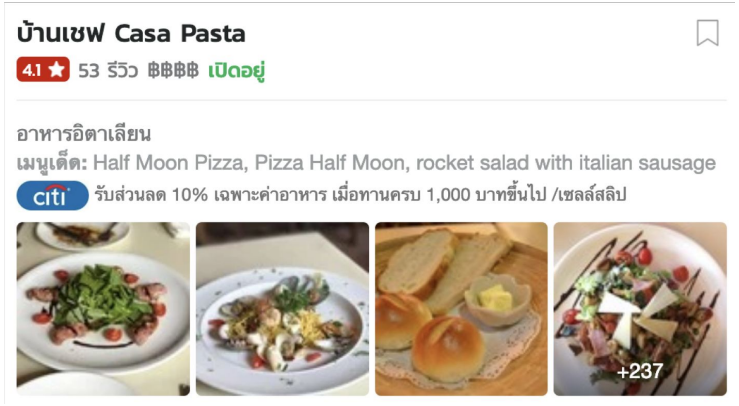
Text classifiers to the rescue

- Build classifiers to add 'attributes' to the search queries and/or documents.
- Where do we get the training set?

Users don't tell you everything

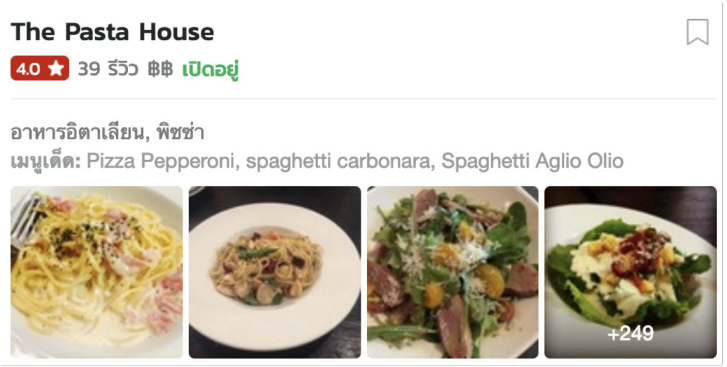
- Distance
- Star ratings
- Open now?

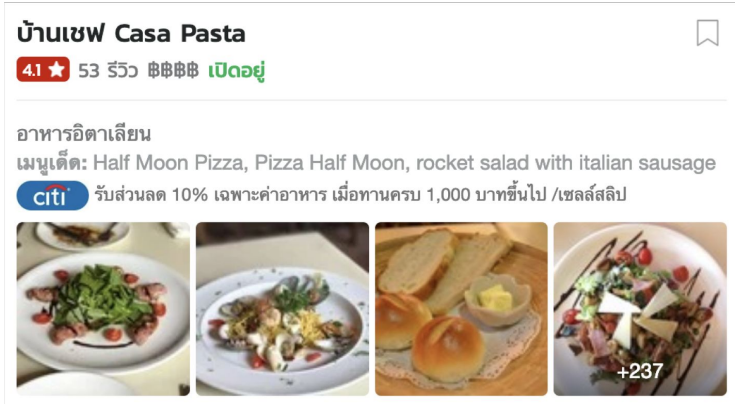




Users don't tell you everything

- Distance
- Star ratings
- Open now?
- Recency
- New restaurant?





Features for search

TermScore(q, d) โดยใช้ TF-IDF

FromScore(q, d)

TitleScore(q, d)

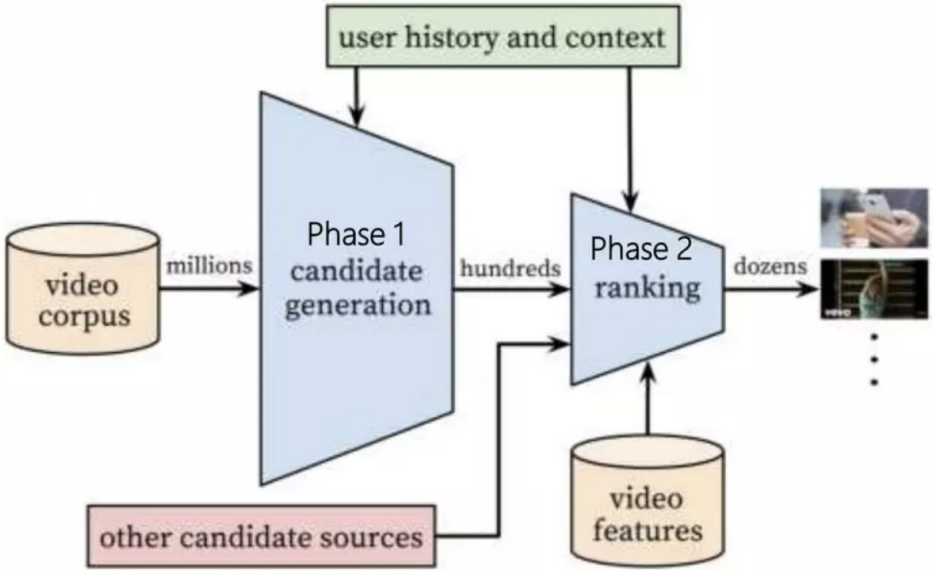
QueryExpansionScore(q, d)

Distance score(u, d)

Recency score(u, d)

Learning to rank

- Use TF-IDF to narrow down the candidates
- Rank the candidates using other sources



Types of features

- Query independent features i.e. computing from document only
- Query dependent features i.e. computing from q and d

ID	Feature Description	Category
1	$\sum_{q_i \in q \cap d} c(q_i, d)$ in body	Q-D
2	$\sum_{q_i \in q \cap d} c(q_i, d)$ in anchor	Q-D
3	$\sum_{q_i \in q \cap d} c(q_i, d)$ in title	Q-D
4	$\sum_{q_i \in q \cap d} c(q_i, d)$ in URL	Q-D
5	$\sum_{q_i \in q \cap d} c(q_i, d)$ in whole document	Q-D
6	$\sum_{q_i \in q} idf(q_i)$ in body	Q
7	$\sum_{q_i \in q} idf(q_i)$ in anchor	Q
8	$\sum_{q_i \in q} idf(q_i)$ in title	Q
9	$\sum_{q_i \in q} idf(q_i)$ in URL	Q
10	$\sum_{q_i \in q} idf(q_i)$ in whole document	Q
11	$\sum_{q_i \in q \cap d} c(q_i, d) \cdot idf(q_i)$ in body	Q-D
12	$\sum_{q_i \in q \cap d} c(q_i, d) \cdot idf(q_i)$ in anchor	Q-D
13	$\sum_{q_i \in q \cap d} c(q_i, d) \cdot idf(q_i)$ in title	Q-D
14	$\sum_{q_i \in q \cap d} c(q_i, d) \cdot idf(q_i)$ in URL	Q-D
15	$\sum_{q_i \in q \cap d} c(q_i, d) \cdot idf(q_i)$ in whole document	Q-D
16	$\|d\|$ of body	D
17	$\|d\|$ of anchor	D
18	$\|d\|$ of title	D
19	$\|d\|$ of URL	D
20	$\|d\|$ of whole document	D
21	BM25 of body	Q-D
22	BM25 of anchor	Q-D
23	BM25 of title	Q-D
24	BM25 of URL	Q-D
25	BM25 of whole document	Q-D
26	LMIR.ABS of body	Q-D
27	LMIR.ABS of anchor	Q-D
28	LMIR.ABS of title	Q-D
29	LMIR.ABS of URL	Q-D
30	LMIR.ABS of whole document	Q-D
31	LMIR.DIR of body	Q-D
32	LMIR.DIR of anchor	Q-D
33	LMIR.DIR of title	Q-D
34	LMIR.DIR of URL	Q-D
35	LMIR.DIR of whole document	Q-D
36	LMIR.JM of body	Q-D
37	LMIR.JM of anchor	Q-D
38	LMIR.JM of title	Q-D
39	LMIR.JM of URL	Q-D
40	LMIR.JM of whole document	Q-D
41	Sitemap based term propagation	Q-D
42	Sitemap based score propagation	Q-D
43	Hyperlink based score propagation: weighted in-link	Q-D
44	Hyperlink based score propagation: weighted out-link	Q-D

45	Hyperlink based score propagation: uniform out-link	Q-D
46	Hyperlink based propagation: weighted in-link	Q-D
47	Hyperlink based feature propagation: weighted out-link	Q-D
48	Hyperlink based feature propagation: uniform out-link	Q-D
49	HITS authority	Q-D
50	HITS hub	Q-D
51	PageRank	D
52	HostRank	D
53	Topical PageRank	Q-D
54	Topical HITS authority	Q-D
55	Topical HITS hub	Q-D
56	Inlink number	D
57	Outlink number	D
58	Number of slash in URL	D
59	Length of URL	D
60	Number of child page	D

Three types of learning to rank algorithms

- Pointwise
- Pairwise
- Listwise

Given a query q and a set of documents $D = (d_1, ..., d_n)$:

Pointwise

Input: Single candidate
$x = (q, d_i)$

Loss: How accurate is the predicted score $s_i \approx y_i$?

Solution: Transform task into Regression.

Pairwise

Input: Pair of candidates
$x_i = (q, d_i)$ and $x_j = (q, d_j)$

Loss: If $y_i > y_j$, then are the predicted scores $s_i > s_j$?

Solution: Transform task into Binary Classification.

Listwise

Input: Whole list of candidates
$x_1 = (q, d_1)$ .. $x_n = (q, d_n)$

Loss: Takes into account position of all retrieved docs.

Solution: Incorporate evaluation metrics (e.g. DCG) into loss.

https://towardsdatascience.com/learning-to-rank-a-complete-guide-to-ranking-using-machine-learning-4c9688d370d4

Pointwise

query	doc id	rank	click?	Term score	Title Score	Recency
1	30	1	1	0.6	0.2	5
1	12	2	0	0.4	0.1	10
1	11	3	1	0.35	0.5	3
1	50	4	0	0.2	0.5	2
2	12	1	0	0.9	0.2	4
2	7	2	0	0.2	0.6	2
2	30	3	0	0.1	0.5	1
2	4	4	1	0.1	0.1	4

$$\sum_{w} tf_{w,Q} \cdot \frac{tf_{w,D}}{tf_{w,D} + \frac{k|D|}{avg|D|}} \cdot \log \frac{|C|}{df_w}$$

Pointwise

query	doc id	rank	click?	Term score	Title Score	Recency
1	30	1	1	0.6	0.2	5
1	12	2	0	0.4	0.1	10
1	11	3	1	0.35	0.5	3
1	50	4	0	0.2	0.5	2
2	12	1	0	0.9	0.2	4
2	7	2	0	0.2	0.6	2
2	30	3	0	0.1	0.5	1
2	4	4	1	0.1	0.1	4

Pairwise

query	doc id	rank	click?	Term score	Title Score	Recency
1	30	1	1	0.6	0.2	5
1	12	2	0	0.4	0.1	10
1	11	3	1	0.35	0.5	3
1	50	4	0	0.2	0.5	2
2	12	1	0	0.9	0.2	4
2	7	2	0	0.2	0.6	2
2	30	3	0	0.1	0.5	1
2	4	4	1	0.1	0.1	4

query	doc id A	doc id B	A>B
1	30	12	1
1	30	11	?
1	30	50	1
1	11	12	1
1	11	50	1

Dense Retrieval

Problem with Classic IR (Sparse keyword-based IR)

- The vocabulary mismatch problem
  - TF-IDF or BM25 cosine similarities only work if there are exact-word overlaps between query and document
  - But the person who wrote the query might not know the exact words might be included in the document!
- Represent both document vectors and query vector with embeddings instead of count vectors

Hypothetical Version of Dense Retrieval: Static embeddings

- Replace TF-IDF vectors with, for example, word2vec
  - Query — Mean of the embeddings of each query word
  - Doc — Mean of the document word embeddings
- Then just compute query-doc cosine as usual
- This is not ideal, because **contextual embeddings** work much better!

Sequence classification with BERT

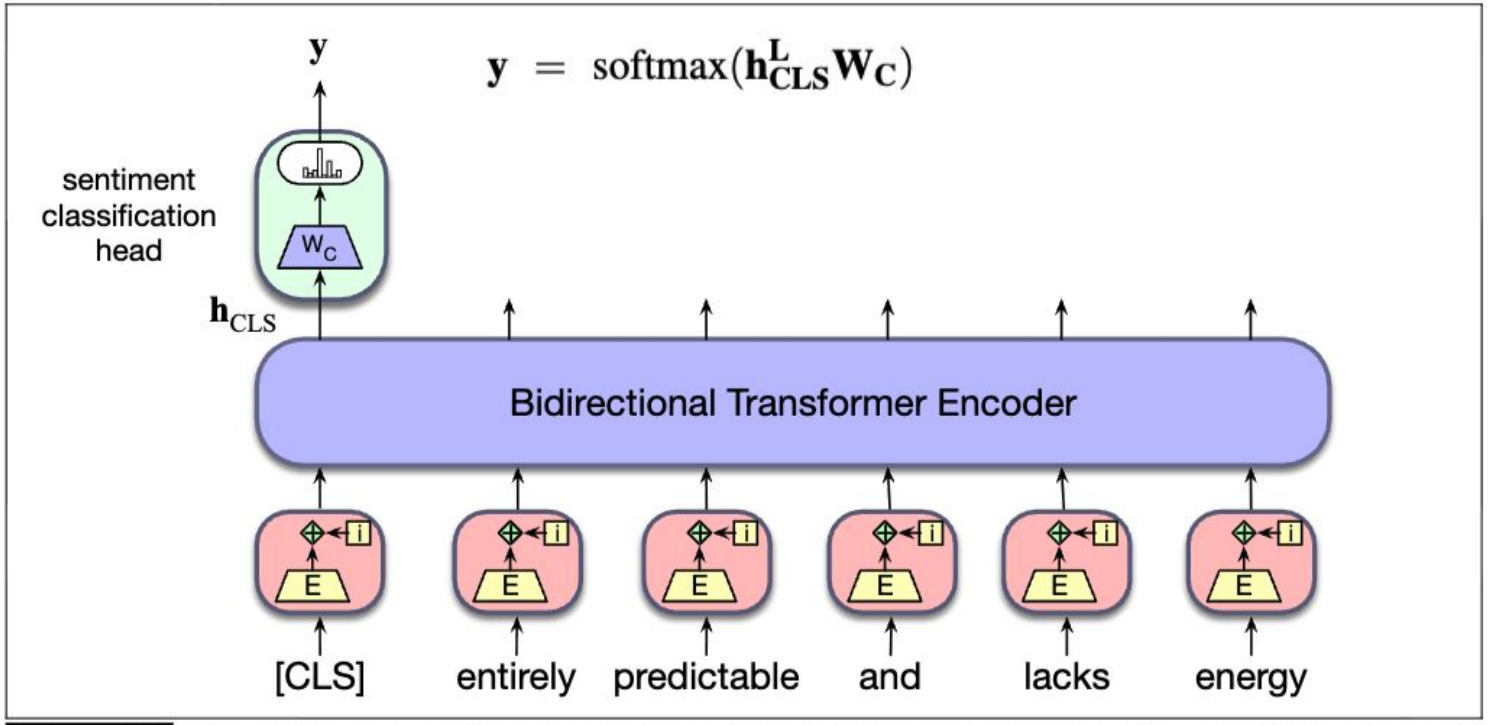
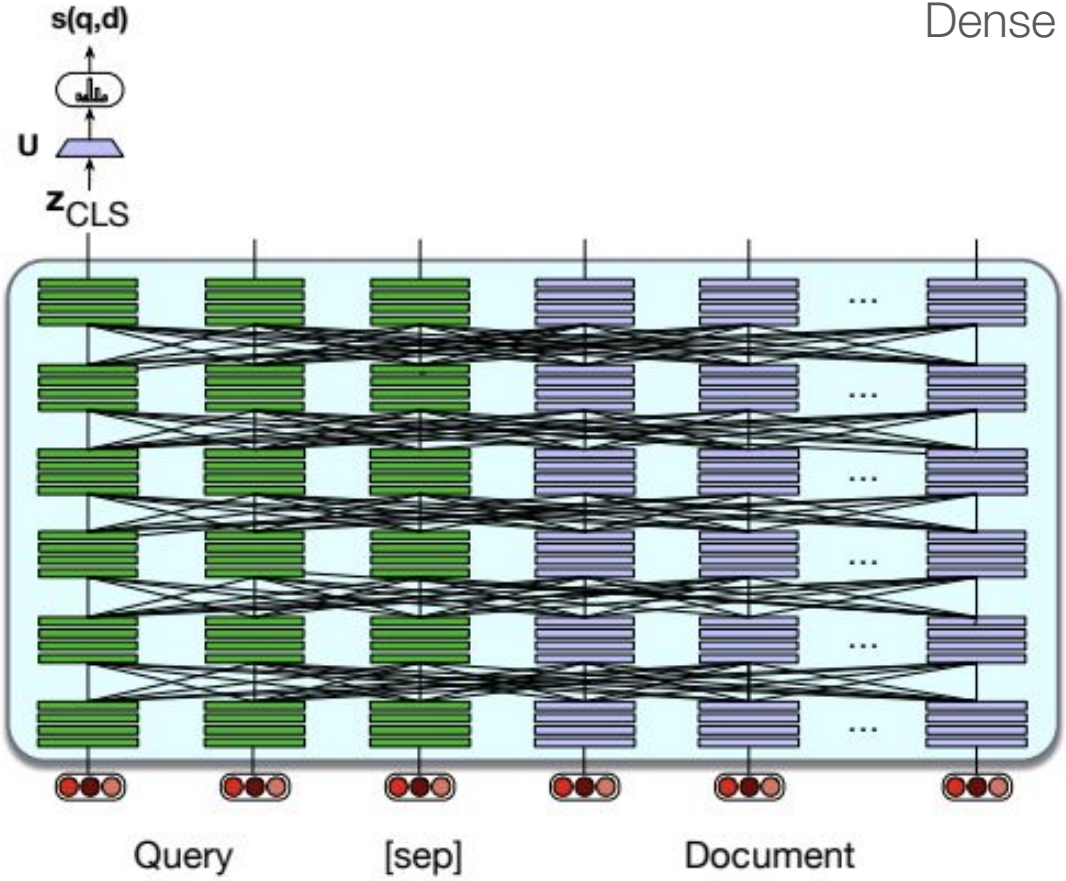


Figure 10.9 Sequence classification with a bidirectional transformer encoder. The output vector for the [CLS] token serves as input to a simple classifier.

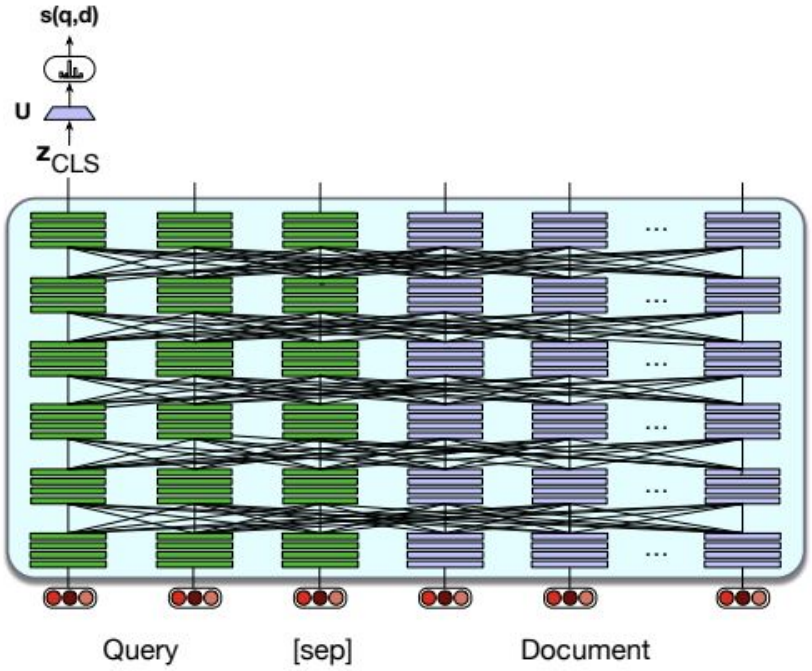
Dense Retrieval #1: Single Encoder



$$\mathbf{z} = \mathrm{BERT}(\mathrm{q}; [\mathrm{SEP}]; \mathrm{d})[\mathrm{CLS}]$$

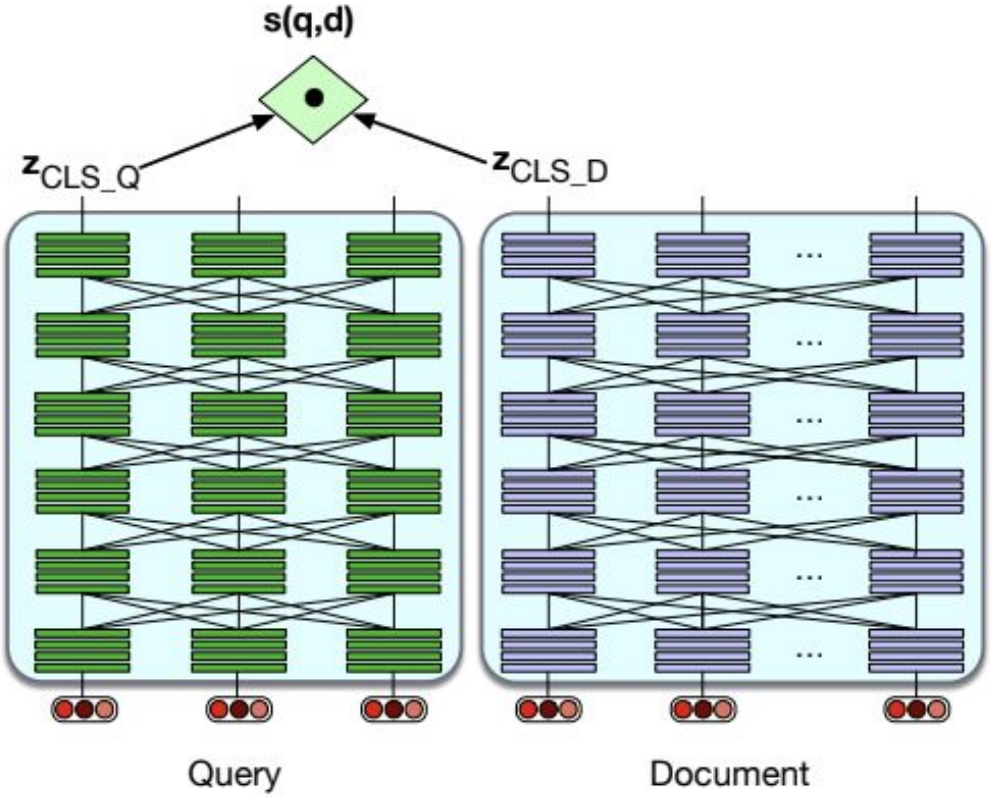
$$\mathrm{score}(q,d) = \mathrm{softmax}(\mathbf{U}(\mathbf{z}))$$

Dense Retrieval #1: Single Encoder



- System is run on passages (e.g. 100 tokens) instead of whole documents
- BERT system + linear layer **U** can then be fine-tuned for the relevance task by gathering a tuning dataset of relevant and non-relevant passages

Dense Retrieval #2: Bi-encoder

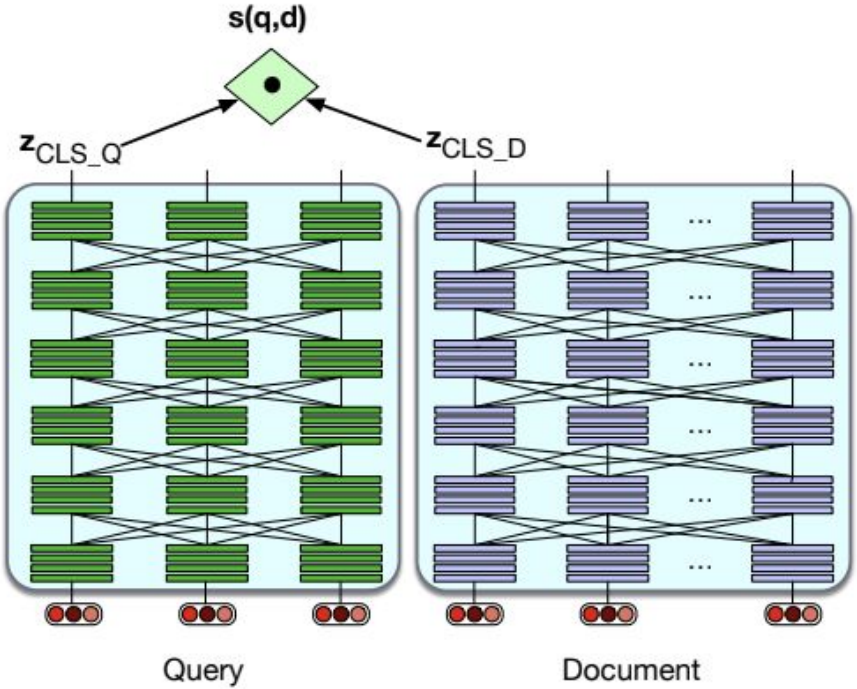


$$\mathbf{z}_q = \mathrm{BERT}_Q(\mathrm{q})[\mathrm{CLS}]$$

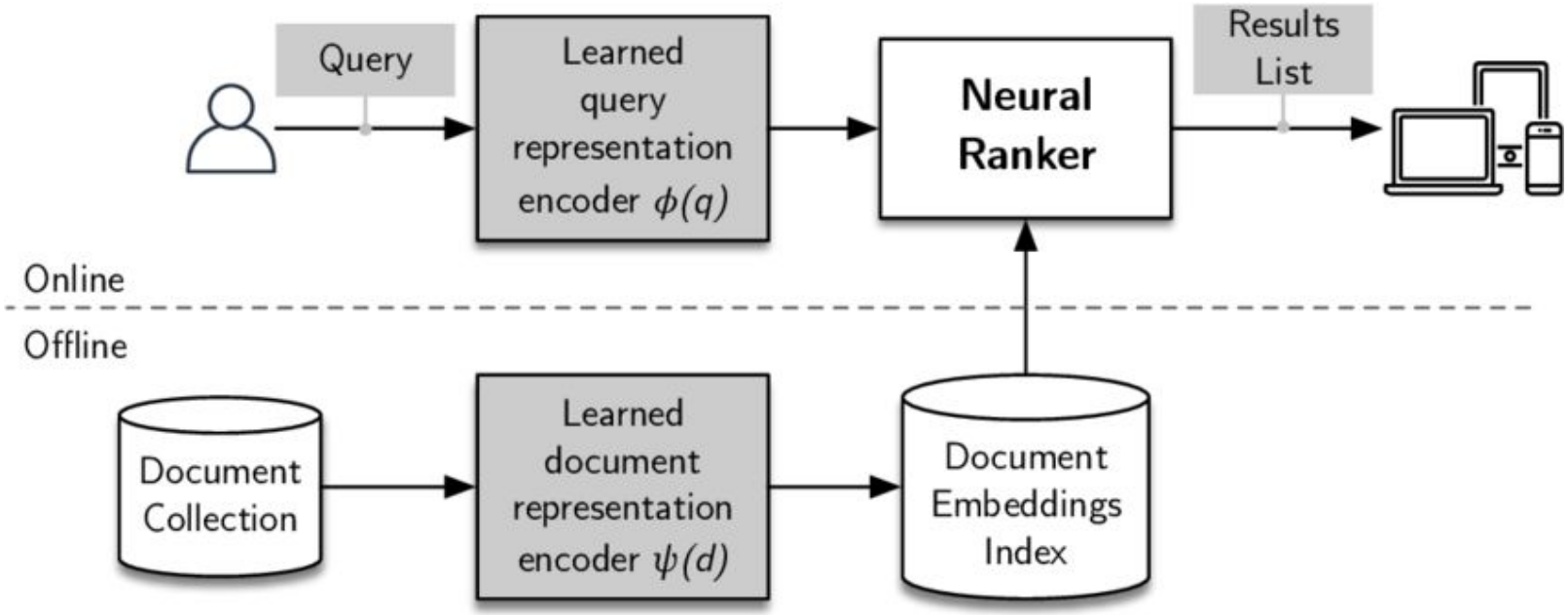
$$\mathbf{z}_d = \mathrm{BERT}_D(\mathrm{d})[\mathrm{CLS}]$$

$$\mathrm{score}(q,d) = \mathbf{z}_q \cdot \mathbf{z}_d$$

Dense Retrieval #2: Bi-encoder



- Encode document vectors in advance, then only encode query when it arrives
- Score is the dot product between query vector and pre-computed doc vector
- Cheaper, more efficient, but less accurate



Embedding-based retrieval

Tonellotto, Nicola. "Lecture notes on neural information retrieval." *arXiv preprint arXiv:2207.13443* (2022).

Gautier Izacard, Mathilde Caron, Lucas Hosseini, Sebastian Riedel, Piotr Bojanowski, Armand Joulin, and Edouard Grave. Unsupervised dense information retrieval with contrastive learning. arXiv preprint arXiv:2112.09118, 2021

Let's play with dense retrieval

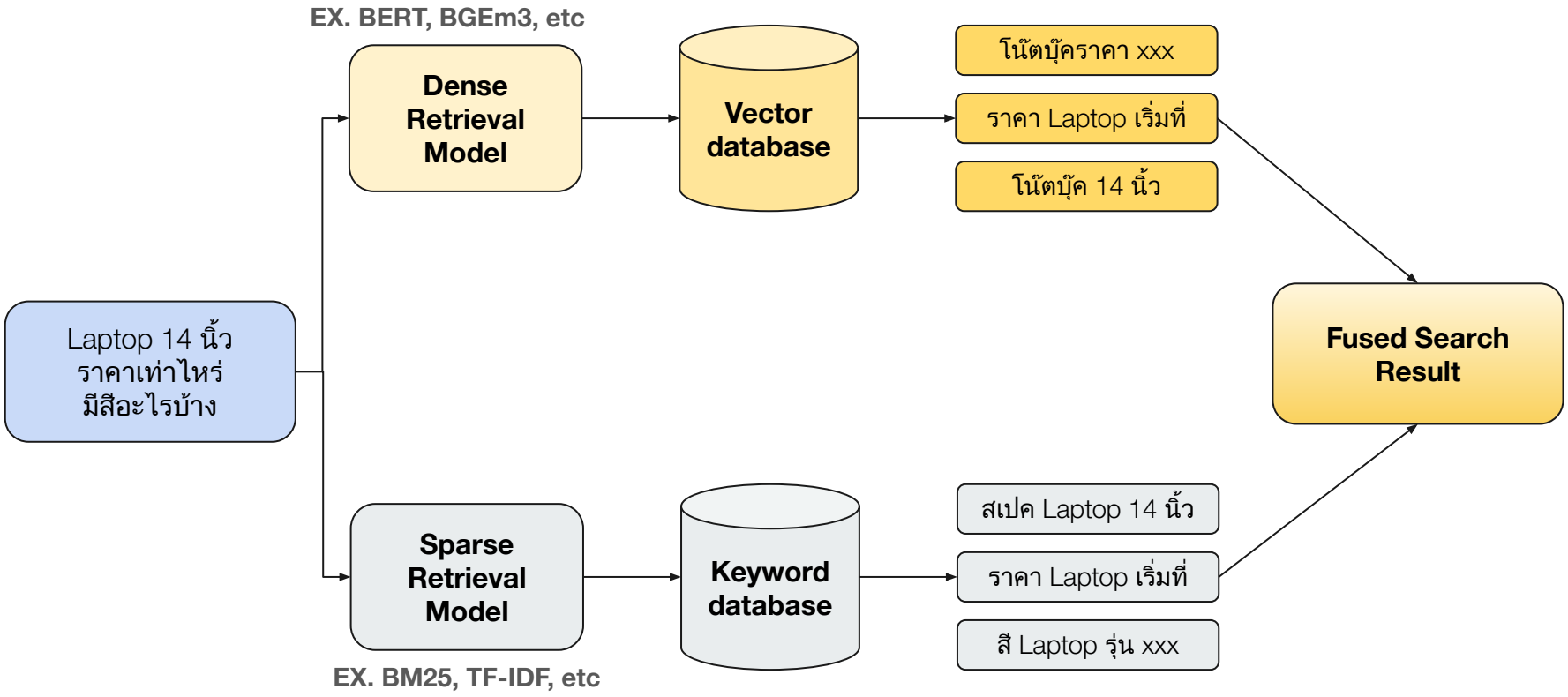
- Dense retrieval can be a lot simpler than sparse retrieval since we do not have to index each token. We store embeddings only.
- BGE-m3 is a general-purpose encoder but highly optimized for search

Training for Dense Retrieval

- To fine-tune the BERT encoders and train the linear layers (and the special [Q] and [D] embeddings)
- On datasets of triplets ⟨q, d+, d−⟩
- Some datasets like MS MARCO Ranking have positive examples

Hybrid Search

- Use spare retrieval methods (e.g. BM25) as first-pass relevance ranking for each document
- Then rerank the top-N ranked documents using expensive methods, e.g. full BERT scoring



Hybrid method

Efficiency in Dense Retrieval

- We must rank every document for its similarity to the query
- Efficiency for sparse word-count vectors — **inverted index**
- Efficiency for dense retrieval
  - **Nearest neighbor search**: finding the set of dense document vectors that have the highest dot product with a dense query vector
  - Approximate Nearest Neighbor Algorithm **Faiss** (Johnson et al., 2017)
    - Approximates the document vector by a smaller quantized vector

Retrieval-Augmented Generation

Questions are a kind of information need

- *Where is the Louvre Museum located?*
- *When was Chiang Mai founded?*
- *What is the price of gasoline today?*
- *Why does the sun rise later in November than in April?*
- *Where in the world do you ship your products?*
- *What do you think of the new governor?*

Just prompt an LLM?

- LLMs seem to store facts in the connections in their feedforward layers
- **...But there are issues!**

AI Overview

The main Louvre Museum is located in **Paris, France, at the Musée du Louvre, 75001 Paris, France**. It is situated on the Right Bank of the Seine River in the city's 1st arrondissement, housed within the historic Louvre Palace.

- **Address:** Rue de Rivoli, 75001 Paris, France.

LLMs hallucinate

- **Hallucination**
  - A response that is not faithful to the facts of the world.
- In the legal domain, LLMs were shown to hallucinate up to 88% of the time (Dahl et al., 2024)

Proprietary data is insufficient

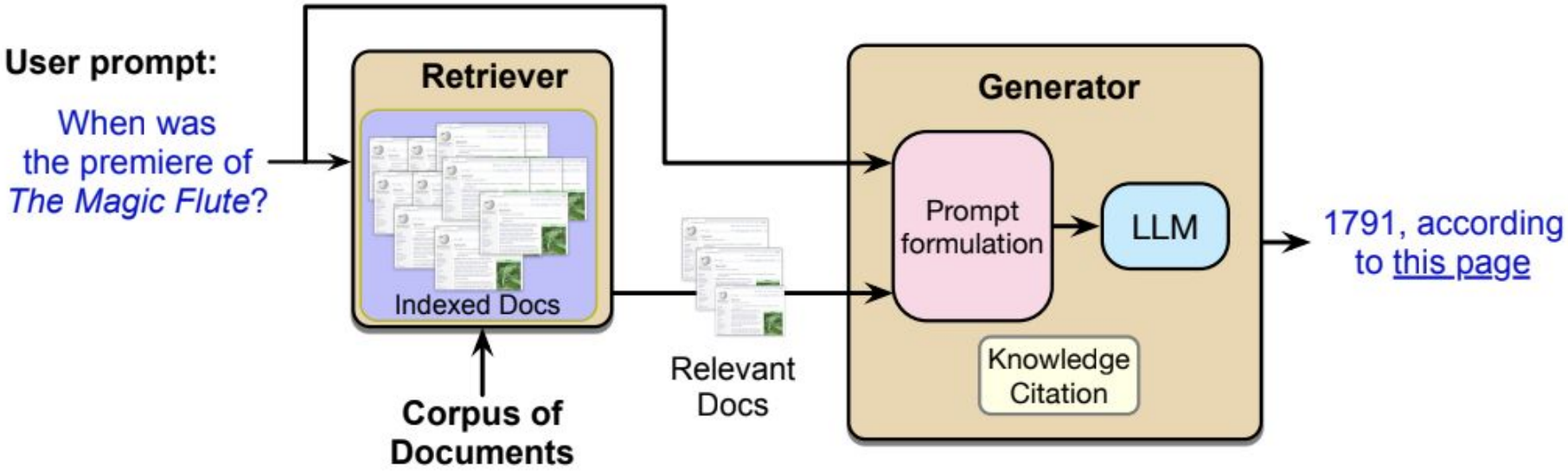
- People need to ask questions about...
  - Personal email
  - Healthcare applications that require medical records
  - Internal corporate documents
  - Legal documents discovery

Dynamic data is not properly handled

- LLMs can't answer questions about rapidly changing information
  - E.g. things that happened last week
- In general, data shifts over time

Solution: IR + LLM = RAG

- **Retrieval-Augmented Generation (RAG)**
  - Uses IR to retrieve documents from some collection
  - Then uses LLM to generate an answer *conditioned on the documents*



Retrieval-Augmented Generation (RAG)

Basic RAG

- Given a document collection **D** and a user query **q**,
  - Call a retriever to return top-k passages
  - Create a prompt that includes **q** and the passages
  - Call an LLM with the prompt

Schematic of a RAG Prompt

```
retrieved passage 1

retrieved passage 2

...

retrieved passage k

Based on these texts, answer this question:  What year
was the premiere of The Magic Flute?
```

Chunking

Chunking

Why do we need to 'chunk'? We need to fit many chunks into the prompt

```
RETRIEVED DOCUMENTS:
(document text)

QUESTION:
(users question)

INSTRUCTIONS:
Answer the users QUESTION using the
DOCUMENT text above.
Keep your answer ground in the facts of
the DOCUMENT.
If the DOCUMENT doesn’t contain the
facts to answer the QUESTION return
{NONE}
```

Chunking Techniques

- Non-overlapping window
- Sliding window

Document

แมวส่งเสียงร้องเพื่อพยายามสื่อสารความต้องการหรือความรู้สึกของตัวเองออกมา ไม่ว่าจะส่งเสียงครางครืด ๆ ในลำคอเวลาที่เราแปรงขนให้ หรือขู่ฟ่อใส่สัตว์ตัวอื่น หากเราสามารถแปลความหมายคร่าว ๆ ของเสียงร้องของแมวได้ เราจะพอเข้าใจในสิ่งที่แมวอยากสื่อสาร

แมวทุกตัวมีความเป็นตัวของตัวเอง นอกจากนี้ วัยก็มีผลต่อการเปล่งเสียงเพื่อสื่อสารด้วยเช่นกัน เช่นลูกแมวมักร้องเหมียวสั้น ๆ เล็ก ๆ น่าเอ็นดู ขณะที่แมวโตจะเงียบกว่า

A B C D | E F G H | I J K L

Non-overlapping window

Chunks (n=10)

*Tokenized by newmm

แมวส่งเสียงร้องเพื่อพยายามสื่อสารความต้องการหรือความรู้สึกของ

ตัวเองออกมา ไม่ว่าจะส่งเสียงครางครืด ๆ

ในลำคอเวลาที่เราแปรงขนให้

หรือขู่ฟ่อใส่สัตว์ตัวอื่น หาก

เราสามารถแปลความหมายคร่าว ๆ ของเสียงร้องของแมวได้

เราจะพอเข้าใจในสิ่งที่แมวอยาก

สื่อสาร

Document

แมวส่งเสียงร้องเพื่อพยายามสื่อสารความต้องการหรือความรู้สึกของตัวเองออกมา ไม่ว่าจะส่งเสียงครางครืด ๆ ในลำคอเวลาที่เราแปรงขนให้ หรือขู่ฟ่อใส่สัตว์ตัวอื่น หากเราสามารถแปลความหมายคร่าว ๆ ของเสียงร้องของแมวได้ เราจะพอเข้าใจในสิ่งที่แมวอยากสื่อสาร

แมวทุกตัวมีความเป็นตัวของตัวเอง นอกจากนี้ วัยก็มีผลต่อการเปล่งเสียงเพื่อสื่อสารด้วยเช่นกัน เช่นลูกแมวมักร้องเหมียวสั้น ๆ เล็ก ๆ น่าเอ็นดู ขณะที่แมวโตจะเงียบกว่า

A B C D D E F G G H I J

Sliding window

Chunks (n=10, windows=5)

แมวส่งเสียงร้องเพื่อพยายามสื่อสารความต้องการหรือความรู้สึกของ

สื่อสารความต้องการหรือความรู้สึกของตัวเองออกมา ไม่ว่า

ตัวเองออกมา ไม่ว่าจะส่งเสียงครางครืด

จะส่งเสียงครางครืด ๆในลำคอเวลา

ๆในลำคอเวลาที่เราแปรงขนให้

ที่เราแปรงขนให้ หรือขู่ฟ่อใส่

หรือขู่ฟ่อใส่สัตว์ตัวอื่น หาก

>>>

Extensions: Agent-based RAG

In agent-based RAG, the LLM is given tools (retriever, database search, web search, calculator, etc.) and it decides when and how to retrieve information while solving the problem. The model can:

a. refuse to answer
b. answer the question right from its own knowledge (no need to search)
c. rewrite the query and retrieve documents from the database
    i. inspect the results and decide whether they are sufficient
        1. yes, answer away
        2. no, rewrite the query and retrieve again
        3. no, refuse to answer

Example agent-based RAG

```
messages = [
    {"role": "system", "content": SYSTEM_PROMPT},
    {"role": "user", "content": user_question}
]

while True:

    response = llm(messages, tools=tools)

    if response.type == "final_answer":
        print(response.content)
        break

    if response.type == "tool_call":

        tool_name = response.tool_name
        tool_args = response.arguments

        result = run_tool(tool_name, tool_args)

        messages.append(response.message)
        messages.append({
            "role": "tool",
            "content": result
        })
```

```
def run_tool(tool_name, tool_args):

    if tool_name == "search_documents":

        query = tool_args["query"]
        top_k = tool_args.get("top_k", 5)
        results = vector_database.search(query, top_k)
        formatted = format_documents(results)
        return formatted


    else:

        return "Error: unknown tool"
```

Prompt

```
You are a question answering assistant with access to external tools.

You should decide the best action before answering a question.

1. If you already know the answer with high confidence, answer directly.
2. If the question requires external information, call the search_documents tool.
3. When calling the search tool, generate a concise query that will help retrieve relevant
documents.
4. After a tool call, you will receive retrieved documents in the tool response.
Carefully read these documents and use them to answer the question.
5. If the documents contain the answer, produce a final answer based on them.
6. If the documents are insufficient, reformulate the search query and retrieve again.
7. If the answer cannot be found after reasonable attempts, refuse to answer rather than
hallucinating.
```

Prompt - Tool calling

```
Available tools:

search_documents(query: string, top_k: integer)
Description: Search the document database and return relevant passages.

Output format:
- If you want to call a tool, return a tool call with the tool name and arguments.
- If you want to answer the user, return the final answer in natural language.
```

Conclusion

- Information retrieval systems weight the information from various text elements and incorporate the non-text metadata by training a model to learn to rank or running A/B testing.
- RAG systems combine IR and LLM to answer questions based on more customized datasets
- Agent-based RAG systems take actions based on the query and can perform tool-calling e.g. using IR to get relevant documents.